PDS ESA — Complete Solved Question Bank

UE20CS901 · Python for Data Science · Every question from all 9 papers, A + B + C, fully solved.

9 papers | 8 sets | 201 questions solved | clean code + brief notes

How to use this: each item shows the **question**, a one-line **note** (the idea / why), and **working code**. Section C code assumes `import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns` and a loaded `df`. Column names follow each paper's data dictionary — verify exact spelling against the real exam CSV with `df.columns`.

May 2025

100 marks · dataset: car_info.csv

SECTION A — THEORY

A1 How are dictionaries different from lists in Python?

> List = ordered, accessed by integer index, allows duplicates. Dict = key→value pairs, accessed by key, keys unique. Dict key lookup is O(1); list search is O(n).

A2 What is a lambda function? How is it different from a regular function?

> A small anonymous one-line function with no name. Single expression, no statements, implicit return. A def function has a name, can hold many statements, and is reusable.

```
double = lambda x: x*2
print(double(5))   # 10
```

↑ Top

A3 What is string immutability in Python?

Once created, a string cannot be changed in place. Any 'edit' returns a NEW string; the original is untouched.

```
s='hi'
# s[0]='H'  -> ERROR
s = 'H' + s[1:]   # new string
```

A4 Slice 'PythonProgramming' to get 'thonPro'.

Use s[start:stop]. Index 2 is 't', stop at 9 (exclusive) gives 'thonPro'.

```
s='PythonProgramming'
print(s[2:9])   # thonPro
```

A5 What does *args do in a function? Provide an example.

*args collects any number of extra positional arguments into a tuple.

```
def total(*args):
    return sum(args)
print(total(1,2,3))  # 6
```

A6 How can sorted() and sort() be used with a list? Example.

list.sort() sorts in place and returns None. sorted(iterable) returns a NEW sorted list and works on anything iterable.

```
a=[3,1,2]
a.sort()            # a is now [1,2,3]
b=sorted([3,1,2])   # b=[1,2,3], original kept
```

A7 How can you get a random number in Python?

Use the random module: random() for float 0–1, randint(a,b) for an integer in [a,b].

```
import random
random.random()       # float 0..1
random.randint(1,6)   # int 1..6
```

A8 What are map and reduce functions in Python?

map(fn, seq) applies fn to every element and returns a map object. reduce(fn, seq) (from functools) folds the sequence into a single value.

↑ Top

```
from functools import reduce
list(map(lambda x:x*x,[1,2,3]))    # [1,4,9]
reduce(lambda a,b:a+b,[1,2,3])     # 6
```

A9 **Difference between reshape() and resize().**

> reshape returns a new array of a compatible shape (total size must match) and does not modify the original. resize changes the array in place and may change total size (pads with repeats/zeros).

```
import numpy as np
a=np.arange(6)
a.reshape(2,3)   # new view, a unchanged
a.resize(3,3)    # a modified in place
```

A10 **Difference between pivot table and cross table.**

> pivot_table aggregates a value column over row/column categories using any aggfunc (mean, sum...). crosstab is a special case that counts frequency of combinations of two categoricals.

```
pd.crosstab(df.A, df.B)
pd.pivot_table(df, index='A', columns='B', values='V', aggfunc='mean')
```

SECTION B — PROGRAMMING

B1 **Return a new list of numbers that are BOTH prime AND palindrome (e.g. 131,7,11); empty list if none.**

> Two tiny helpers: prime test (check divisors up to sqrt), palindrome test (string equals its reverse). Filter with a comprehension.

```
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0: return False
    return True

def is_pal(n):
    return str(n) == str(n)[::-1]

def prime_palindromes(nums):
    return [x for x in nums if is_prime(x) and is_pal(x)]

print(prime_palindromes([7,11,12,131,23,101]))  # [7,11,131,101]
```

B2 **Student marks system: add student, update marks, average of all, display all.**

> A dict name→marks plus four small functions. Same CRUD skeleton reused across many papers.

↑ Top

```
students = {}
def add_student(name, marks):  students[name] = marks
def update_marks(name, marks):
    if name in students: students[name] = marks
def average_marks():
    return sum(students.values())/len(students) if students else 0
def display():
    for n, m in students.items(): print(n, '->', m)

add_student('Asha', 88); add_student('Ravi', 72)
update_marks('Ravi', 80); print(average_marks()); display()
```

B3 **Adjust employee bonuses by experience tier (>10y: +20%+3%*base*rating; 5–10y: +15%+2%*...; <5y: +10%+1%*...). Return new dict.**

> Loop the dict, branch on years, build a NEW dict so the original is preserved. Only the percentages change between papers.

```
def adjust_bonus(emps):   # emps: name -> (years, base_bonus, rating)
    out = {}
    for name, (yrs, base, rating) in emps.items():
        if yrs > 10:   out[name] = base*1.20 + 0.03*base*rating
        elif yrs >= 5: out[name] = base*1.15 + 0.02*base*rating
        else:          out[name] = base*1.10 + 0.01*base*rating
    return out

emps = {'A':(12,1000,5), 'B':(7,1000,3), 'C':(2,1000,4)}
print(adjust_bonus(emps))
```

B4 **From a list of sentences, dict of unique words (case-insensitive, length>=4) -> counts. Ignore punctuation.**

> Lowercase, pull alphabetic words with regex, keep length>=4, count with the .get(k,0)+1 idiom.

```
import re
def word_counts(sentences):
    counts = {}
    for s in sentences:
        for w in re.findall(r'[a-zA-Z]+', s.lower()):
            if len(w) >= 4:
                counts[w] = counts.get(w, 0) + 1
    return counts

print(word_counts(['The quick brown Fox', 'the lazy dog runs']))
```

B5 **Given names + DOBs ('YYYY-MM-DD'): (i) youngest student; (ii) students born in December.**

> Parse dates with datetime. Youngest = max date. December = month == 12.

↑ Top

```
from datetime import datetime
names = ['Asha','Ravi','Meena']
dobs  = ['2001-12-05','1999-06-20','2003-03-11']
pairs = [(n, datetime.strptime(d, '%Y-%m-%d')) for n, d in zip(names, dobs)]
youngest = max(pairs, key=lambda p: p[1])[0]
print('Youngest:', youngest)
print('Born in Dec:', [n for n, d in pairs if d.month == 12])
```

SECTION C — DATA ANALYSIS (EDA)

C1 (i) Load car_info; count cars priced > $25,000; average mileage of those. Inference.

Filter then aggregate. Always finish with a one-line inference (separately marked).

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns
df = pd.read_csv('car_info.csv')      # INSERT PATH
exp = df[df['Price'] > 25000]
print('Count:', len(exp))
print('Avg mileage:', exp['Mileage'].mean())
# Inference: pricier cars usually show lower mileage.
```

C2 (ii) Separate Petrol vs Diesel; average price of each group.

Boolean-mask into two sub-DataFrames, then .mean() on each.

```
petrol = df[df['Fuel_Type'] == 'Petrol']
diesel = df[df['Fuel_Type'] == 'Diesel']
print('Petrol avg price:', petrol['Price'].mean())
print('Diesel avg price:', diesel['Price'].mean())
```

C3 (iii) Average price and maximum mileage for each make.

Single groupby with a named aggregation.

```
summary = df.groupby('Make').agg(avg_price=('Price','mean'),
                                 max_mileage=('Mileage','max'))
print(summary)
```

C4 (iv) Add Mileage_Category: <30000 Low, 30000–100000 Moderate, >100000 High.

Write a function for one value, then .apply over the column.

```
def cat(m):
    if m < 30000: return 'Low'
    elif m < 100000: return 'Moderate'
    return 'High'
```

↑ Top

```
df['Mileage_Category'] = df['Mileage'].apply(cat)
print(df[['Mileage','Mileage_Category']].head())
```

C5 **(v) Highest-priced car for every make.**

> Sort then drop_duplicates on Make, or use idxmax per group.

```
idx = df.groupby('Make')['Price'].idxmax()
print(df.loc[idx, ['Make','Price']])
```

C6 **(b-i) Boxplot of price per Make grouped by Fuel_Type, with title.**

> seaborn boxplot with hue. Remember the title and rotate x labels.

```
sns.boxplot(data=df, x='Make', y='Price', hue='Fuel_Type')
plt.title('Price Distribution by Make and Fuel Type')
plt.xticks(rotation=45); plt.tight_layout(); plt.show()
```

C7 **(b-ii) Average price per make, filtering out missing Price or Mileage.**

> dropna on the two columns first, then groupby.

```
clean = df.dropna(subset=['Price','Mileage'])
print(clean.groupby('Make')['Price'].mean())
```

C8 **(b-iii) Distribution of Fuel_Type across Make; most common fuel per brand.**

> crosstab Make×Fuel gives counts; idxmax(axis=1) gives the top fuel per make.

```
ct = pd.crosstab(df['Make'], df['Fuel_Type'])
print(ct)
print('Most common fuel per make:'); print(ct.idxmax(axis=1))
ct.plot(kind='bar', stacked=True); plt.title('Fuel Type by Make'); plt.show()
```

C9 **(b-iv) Do prices vary with age? Differences between manual and automatic? Plots + comment.**

> Scatter price vs age, colored by transmission. Comment on both trends.

```
sns.scatterplot(data=df, x='Age', y='Price', hue='Transmission')
plt.title('Price vs Age by Transmission'); plt.show()
# Comment: price falls as age rises; automatics often hold price better.
```

C10 **(b-v) Which make is most listed? Support with a plot.**

↑ Top

value_counts then a bar/countplot.

```
print(df['Make'].value_counts().head(1))
sns.countplot(data=df, y='Make', order=df['Make'].value_counts().index)
plt.title('Listings per Make'); plt.show()
```

October 2024

100 marks · dataset: car_info.csv

SECTION A — THEORY

A1 How to generate a random float between 0 and 1?

random.random() returns a float in [0.0, 1.0). numpy: np.random.rand().

```
import random; print(random.random())
```

A2 Difference between pop() and remove() in a list.

pop(index) removes by position and RETURNS the item (default last). remove(value) deletes the first matching value and returns nothing.

```
a=[10,20,30]; a.pop(1)   # returns 20
b=[10,20,30]; b.remove(20)
```

A3 Purpose of filter(); filter even numbers from a list.

filter(fn, seq) keeps only elements where fn returns True.

```
evens = list(filter(lambda x: x%2==0, [1,2,3,4,5,6]))  # [2,4,6]
```

A4 Purpose of enumerate(); example in a loop.

enumerate gives (index, value) pairs so you get the position while looping.

```
for i, v in enumerate(['a','b','c']):
    print(i, v)
```

A5 Difference between arange() and linspace() in NumPy.

arange(start,stop,step) is defined by step size (stop exclusive). linspace(start,stop,n) gives n evenly spaced points including the endpoint.

↑ Top

```
np.arange(0,1,0.25)     # [0,0.25,0.5,0.75]
np.linspace(0,1,5)      # [0,0.25,0.5,0.75,1]
```

A6 Select a subset of rows by condition in a DataFrame. Example.

Use a boolean mask inside df[...].

```
df[df['Age'] > 30]
```

A7 Explain groupby in pandas with syntax + an aggregate example.

Split rows by a category, apply an aggregate, combine results.

```
df.groupby('Dept')['Salary'].mean()
```

A8 How to remove missing data from a DataFrame?

dropna() removes rows/columns with NaN; pass subset to target columns; axis=1 for columns.

```
df.dropna()                 # drop any row with NaN
df.dropna(subset=['Price'])
```

A9 Difference between apply() and map() in pandas.

map works element-wise on a Series only. apply works on a Series or across DataFrame rows/columns and is more flexible.

```
df['c'].map(str.upper)
df.apply(lambda row: row['a']+row['b'], axis=1)
```

A10 How to reset the index of a DataFrame? Example.

reset_index(drop=True) renumbers rows 0..n and discards the old index.

```
df = df.reset_index(drop=True)
```

SECTION B — PROGRAMMING

B1 Return a new list of only the prime numbers from a list; empty if none.

Same prime test as elsewhere, filtered by comprehension.

```
def is_prime(n):
    if n < 2: return False
```

↑ Top

```
    for i in range(2, int(n**0.5)+1):
        if n % i == 0: return False
    return True
print([x for x in [2,4,7,9,11,15] if is_prime(x)])  # [2,7,11]
```

B2 **Inventory system: add item (name,qty,price); update qty; total value (sum qty*price); display all.**

> Dict name→(qty, price). Same CRUD skeleton; total value multiplies and sums.

```
inv = {}
def add_item(name, qty, price): inv[name] = (qty, price)
def update_qty(name, qty):
    if name in inv: inv[name] = (qty, inv[name][1])
def total_value():
    return sum(q*p for q, p in inv.values())
def display():
    for n,(q,p) in inv.items(): print(n, q, p)

add_item('Pen',10,5); add_item('Book',3,50)
print(total_value()); display()
```

B3 **Adjust salaries by experience+salary thresholds (tiered % rules); others unchanged. Return new dict.**

> Same tiered pattern as the bonus question, with an extra salary-ceiling condition per tier.

```
def adjust(emps):   # name -> (years, salary, rating)
    out = {}
    for n,(y,s,r) in emps.items():
        if y>10 and s<70000:   out[n]=s*1.15 + 0.02*s*r
        elif 5<=y<=10 and s<50000: out[n]=s*1.10 + 0.015*s*r
        elif y<5 and s<40000:  out[n]=s*1.05 + 0.01*s*r
        else: out[n]=s          # no change
    return out
```

B4 **Word counts (len>=4, case-insensitive) AND a histogram of word lengths.**

> Same counting idiom, then plot the lengths of the unique words.

```
import re, matplotlib.pyplot as plt
def analyze(sentences):
    counts = {}
    for s in sentences:
        for w in re.findall(r'[a-zA-Z]+', s.lower()):
            if len(w)>=4: counts[w]=counts.get(w,0)+1
    lengths=[len(w) for w in counts]
```

↑ Top

```
    plt.hist(lengths, bins=range(4, max(lengths)+2)); plt.title('Word lengths'); pl
    return counts
```

B5 longest_consecutive_sequence(nums): length of longest run of consecutive integers.

Put numbers in a set; for each number with no predecessor, count upward. O(n).

```
def longest_consecutive_sequence(nums):
    s=set(nums); best=0
    for x in s:
        if x-1 not in s:           # start of a run
            y=x
            while y+1 in s: y+=1
            best=max(best, y-x+1)
    return best
print(longest_consecutive_sequence([100,4,200,1,3,2]))  # 4
```

SECTION C — DATA ANALYSIS (EDA)

C1 (i) Count cars priced > $25,000; average mileage; inference.

Identical to May'25 — filter then aggregate.

```
df = pd.read_csv('car_info.csv')
exp = df[df['Price']>25000]
print(len(exp), exp['Mileage'].mean())
```

C2 (ii) Two DataFrames for Petrol and Diesel; show average price each.

Mask into two frames; mean each.

```
petrol=df[df['Fuel_Type']=='Petrol']; diesel=df[df['Fuel_Type']=='Diesel']
print(petrol['Price'].mean(), diesel['Price'].mean())
```

C3 (iii) Average price and maximum mileage per make.

groupby + agg.

```
print(df.groupby('Make').agg(avg=('Price','mean'), mx=('Mileage','max')))
```

C4 (iv) Mileage category Low/Moderate/High.

apply a banding function.

```
def cat(m): return 'Low' if m<30000 else 'Moderate' if m<100000 else 'High'
```

↑ Top

```
df['Cat']=df['Mileage'].apply(cat)
```

C5 **(v) Most expensive car for each make.**

> idxmax per group.

```
print(df.loc[df.groupby('Make')['Price'].idxmax(), ['Make','Price']])
```

C6 **(b-i) Relationship between Price and Age across fuel types; comment on pricing across mileage ranges.**

> Scatter / lineplot price vs age with fuel hue.

```
sns.scatterplot(data=df,x='Age',y='Price',hue='Fuel_Type'); plt.show()
```

C7 **(b-ii) Average price per make filtering out missing Price/Mileage.**

> dropna then groupby.

```
print(df.dropna(subset=['Price','Mileage']).groupby('Make')['Price'].mean())
```

C8 **(b-iii) Distribution of Fuel_Type across Make; most popular fuel per brand.**

> crosstab + idxmax.

```
ct=pd.crosstab(df['Make'],df['Fuel_Type']); print(ct.idxmax(axis=1))
```

C9 **(b-iv) Price vs age; manual vs automatic differences; plots + comment.**

> Scatter with transmission hue.

```
sns.scatterplot(data=df,x='Age',y='Price',hue='Transmission'); plt.show()
```

C10 **(b-v) Most-listed make; support with plot.**

> value_counts + countplot.

```
sns.countplot(data=df,y='Make',order=df['Make'].value_counts().index); plt.show()
```

↑ Top

November 2023

100 marks · dataset: water.csv, titanic.csv

SECTION A — THEORY

A1 What is type conversion in Python?

Changing a value from one type to another: explicit (int('5'), str(9)) or implicit (Python auto-promotes int+float to float).

```
int('5'); float(3); str(10)
```

A2 What are *args and **kwargs?

*args = extra positional args as a tuple; **kwargs = extra keyword args as a dict.

```
def f(*args, **kwargs): print(args, kwargs)
```

A3 Difference between del(), clear(), remove(), pop().

del removes by index/slice (statement); clear() empties the list; remove(v) deletes first matching value; pop(i) removes by index and returns it.

```
a=[1,2,3]; a.pop(); a.remove(1); a.clear(); del a
```

A4 What are map and reduce functions?

map applies a function to each element; reduce (functools) folds the sequence to one value.

```
from functools import reduce; reduce(lambda a,b:a+b,[1,2,3])
```

A5 What is Dictionary and List comprehension?

Concise one-line construction. List: [expr for x in seq]. Dict: {k:v for ... }.

```
[x*x for x in range(5)]
{c:ord(c) for c in 'abc'}
```

A6 How to combine different DataFrames in pandas?

pd.concat to stack; pd.merge for SQL-style joins on keys; df.join on index.

```
pd.concat([d1,d2]); pd.merge(d1,d2,on='id')
```

↑ Top

A7 Identify and deal with missing values in a DataFrame.

Detect with isnull().sum(). Handle by dropna() or fillna(mean/median/mode).

```
df.isnull().sum(); df.fillna(df.mean(numeric_only=True))
```

A8 Explain ranking methods for Series.

rank() assigns ranks; method= average (default), min, max, first, dense controls tie handling.

```
s.rank(method='dense')
```

A9 Difference between reshape() and resize().

reshape returns a new compatible-size array (original unchanged); resize modifies in place and may change total size.

```
a.reshape(2,3); a.resize(3,3)
```

A10 Difference between sort_values() and sort_index().

sort_values orders by the data; sort_index orders by index labels.

```
s.sort_values(); s.sort_index()
```

SECTION B — PROGRAMMING

B1 Movie age-limit checker + UsernameValidation(str) per notebook rules.

Username rule pattern: length window, starts with letter, only letters/digits/underscore, no trailing underscore.

```
def UsernameValidation(s):
    return (4 <= len(s) <= 25 and s[0].isalpha()
            and s.replace('_','').isalnum() and not s.endswith('_'))
print(UsernameValidation('user_1'))  # True
```

B2 telephone_match(s): match a string against a phone-keypad digit mapping.

Map each letter to its phone digit, translate the string, compare. Core idea: a dict letter->digit.

```
keypad = {**dict.fromkeys('abc','2'), **dict.fromkeys('def','3'),
          **dict.fromkeys('ghi','4'), **dict.fromkeys('jkl','5'),
          **dict.fromkeys('mno','6'), **dict.fromkeys('pqrs','7'),
          **dict.fromkeys('tuv','8'), **dict.fromkeys('wxyz','9')}
```

↑ Top

```
def to_digits(word): return ''.join(keypad.get(c,c) for c in word.lower())
print(to_digits('hello'))  # 43556
```

B3 Calculate strength of a password (per notebook rules).

> Score by checks: length>=8, has upper, lower, digit, special. More checks passed = stronger.

```
import re
def strength(p):
    score = sum([len(p)>=8, bool(re.search(r'[A-Z]',p)),
                 bool(re.search(r'[a-z]',p)), bool(re.search(r'[0-9]',p)),
                 bool(re.search(r'[^A-Za-z0-9]',p))])
    return ['Very Weak','Weak','Medium','Strong','Very Strong','Excellent'][score]
print(strength('Abc12!xy'))
```

B4 Collatz conjecture for a starting value (sequence on one line).

> Even -> n/2, odd -> 3n+1, stop at 1. Collect steps in a list.

```
def collatz(n):
    seq=[n]
    while n!=1:
        n = n//2 if n%2==0 else 3*n+1
        seq.append(n)
    return seq
print(*collatz(6))
```

SECTION C — DATA ANALYSIS (EDA)

C1 (a1) Water: samples with TDS>500; average hardness of those; inference.

> Filter then mean; one-line inference.

```
df=pd.read_csv('water.csv')
hi=df[df['TDS']>500]
print(len(hi), hi['Hardness'].mean())
```

C2 (a2) Two DataFrames potable vs non-potable; show range (min,max) per parameter.

> Mask by Potability, then agg min/max.

```
pot=df[df['Potability']==1]; npot=df[df['Potability']==0]
print(pot.agg(['min','max']))
```

C3 (a3) Average, min, max turbidity for each potability status.

↑ Top

groupby Potability, agg the three stats.

```
print(df.groupby('Potability')['Turbidity'].agg(['mean','min','max']))
```

C4 **(a4) Category column for Hardness: Low/Moderate/High.**

apply a banding function (thresholds per dataset).

```
def hc(h): return 'Low' if h<100 else 'Moderate' if h<200 else 'High'
df['Hardness_Cat']=df['Hardness'].apply(hc)
```

C5 **(a5) Samples that are potable OR hardness>200.**

Combine masks with | (remember parentheses).

```
print(df[(df['Potability']==1) | (df['Hardness']>200)])
```

C6 **(b1) Titanic: survival proportion per embarked port; plot.**

groupby Embarked, mean of Survived; bar plot.

```
t=pd.read_csv('titanic.csv')
print(t.groupby('Embarked')['Survived'].mean())
t.groupby('Embarked')['Survived'].mean().plot(kind='bar'); plt.show()
```

C7 **(b2) Youngest and oldest passengers.**

min/max of Age (or idxmin/idxmax for the row).

```
print(t['Age'].min(), t['Age'].max())
```

C8 **(b3) Count passengers per age group (children/adults/seniors).**

Bin Age with pd.cut, then value_counts.

```
t['AgeGroup']=pd.cut(t['Age'],[0,12,60,120],labels=['Child','Adult','Senior'])
print(t['AgeGroup'].value_counts())
```

C9 **(b4) Group by Embarked: count, average age, median fare per port.**

groupby with a dict of aggregations.

↑ Top

```
print(t.groupby('Embarked').agg(n=('PassengerId','count'),
      avg_age=('Age','mean'), med_fare=('Fare','median')))
```

C10 **(b5) family_size column via get_family_size + lambda over rows.**

Define function (sibsp+parch+1), apply row-wise with lambda.

```
def get_family_size(sib,par): return sib+par+1
t['family_size']=t.apply(lambda r: get_family_size(r['SibSp'],r['Parch']), axis=1)
```

C11 **(b6) Extract cabin type from Cabin; visualize survival rates; plot.**

First letter of Cabin = deck; groupby that, mean survival; bar plot.

```
t['Deck']=t['Cabin'].astype(str).str[0]
t.groupby('Deck')['Survived'].mean().plot(kind='bar'); plt.show()
```

March 2024 (ESA)

100 marks · dataset: athletes.csv, regions.csv

SECTION A — THEORY

A1 **What is type casting? Explain with respect to Python.**

Converting a value's type. Explicit via int(), float(), str(); implicit when Python auto-promotes (int+float -> float).

```
int('7'); float(2); str(99)
```

A2 **Difference between tuples and lists.**

List mutable (), changeable, []; tuple immutable, () , hashable, slightly faster.

```
[1,2].append(3)   # ok
(1,2)[0]=9        # ERROR
```

A3 **Lambda that prints the sum of [5,8,10,20,50,100].**

Lambda wraps sum over the list.

```
total = lambda lst: sum(lst)
print(total([5,8,10,20,50,100]))  # 193
```

↑ Top

A4 What is a string and explain slicing.

An immutable ordered sequence of characters. Slice s[start:stop:step]; negative indexes count from the end.

```
'Python'[1:4]   # 'yth'
'Python'[::-1]  # reversed
```

A5 What are *args and **kwargs?

*args = tuple of extra positional args; **kwargs = dict of extra keyword args.

```
def f(*a, **k): pass
```

A6 Difference between pivot table and cross table.

pivot_table aggregates a value over categories with any aggfunc; crosstab counts frequencies of two categoricals.

```
pd.pivot_table(df,index='A',values='V',aggfunc='mean')
```

A7 How can you get a random number in Python?

random.random() float; random.randint(a,b) int.

```
import random; random.randint(1,100)
```

A8 What are map and reduce functions?

map applies fn to each element; reduce folds to one value.

```
list(map(str,[1,2,3]))
```

A9 How is vstack() different from hstack()?

vstack stacks arrays as rows (vertical); hstack joins them as columns (horizontal).

```
np.vstack([a,b]); np.hstack([a,b])
```

A10 How can sorted() and sort() be used with a list?

sort() in place returns None; sorted() returns a new list.

```
a.sort(); b=sorted(a)
```

↑ Top

SECTION B — PROGRAMMING

B1 **Stock investing: compute purchase cost, buy commission, sale price, sale commission, profit margin.**

Plain arithmetic; commission = rate * transaction value; profit = net sale - net cost.

```
units, buy, sell, comm = 2000, 40, 42.75, 0.03
cost = units*buy; buy_comm = cost*comm
revenue = units*sell; sell_comm = revenue*comm
profit = (revenue - sell_comm) - (cost + buy_comm)
print(cost, buy_comm, revenue, sell_comm, profit)
```

B2 **product_inventory dict; update_inventory(inv,shipment); apply_discount(inv,discounts).**

Dict updates: .update() merges shipment quantities; discount function overwrites prices.

```
product_inventory = {'pen':10}
def update_inventory(inv, shipment):
    inv.update(shipment); return inv
def apply_discount(inv, discounts):
    for k,price in discounts.items():
        if k in inv: inv[k]=price
    return inv
```

B3 **find_special_codes(str): return alphanumeric words with no special characters.**

Split on whitespace, keep tokens that are purely alphanumeric.

```
def find_special_codes(s):
    return [w for w in s.split() if w.isalnum()]
print(find_special_codes('abc12 x@y 7h7 !!'))  # ['abc12','7h7']
```

B4 **map()+filter(): list of squares of even numbers in 1..10.**

filter evens, map to squares.

```
print(list(map(lambda x:x*x, filter(lambda x:x%2==0, range(1,11)))))
```

B5 **Fibonacci via list comprehension, comma-separated, n from user.**

Comprehensions can't truly self-reference; build iteratively then join. (Examiner accepts a generator/loop.)

```
def fib(n):
    a,b,out=0,1,[]
    for _ in range(n): out.append(a); a,b=b,a+b
```

↑ Top

```
    return out
print(','.join(map(str, fib(10))))
```

B6 **Enter 10 numbers (1–15); replace entries >10 with 10 using map().**

map with a capping lambda.

```
nums=[3,12,7,15,9,11,2,14,5,10]
print(list(map(lambda x: 10 if x>10 else x, nums)))
```

SECTION C — DATA ANALYSIS (EDA)

C1 **1. Merge athletes and regions.**

pd.merge on the shared region/NOC key.

```
ath=pd.read_csv('athletes.csv'); reg=pd.read_csv('regions.csv')
df=pd.merge(ath, reg, on='NOC', how='left')
```

C2 **2. Sport with the most Gold medals (Top 5).**

Filter Medal=='Gold', groupby Sport, count, top 5.

```
g=df[df['Medal']=='Gold']
print(g['Sport'].value_counts().head(5))
```

C3 **3. Total Female athletes in each Summer Olympics; plot.**

Filter Sex=='F' & Season=='Summer', groupby Year, nunique of names; bar plot.

```
f=df[(df['Sex']=='F') & (df['Season']=='Summer')]
out=f.groupby('Year')['Name'].nunique()
out.plot(kind='bar'); plt.title('Female athletes per Summer Games'); plt.show()
```

C4 **4. Male vs female participation over time (Summer & Winter); line charts.**

groupby Year+Sex (per season), count, unstack, plot lines.

```
for s in ['Summer','Winter']:
    sub=df[df['Season']==s]
    p=sub.groupby(['Year','Sex'])['Name'].nunique().unstack()
    p.plot(); plt.title(s); plt.show()
```

C5 **5. Year India won its first Summer Olympics Gold.**

↑ Top

Filter India + Gold + Summer, take min Year.

```
ind=df[(df['region']=='India') & (df['Medal']=='Gold') & (df['Season']=='Summer')]
print(ind['Year'].min())
```

C6 **6. Category column: Tall/Short & Heavy/Light (H>=1.8, W>=80).**

apply a row function combining two thresholds.

```
def cat(r):
    t='Tall' if r['Height']>=1.8 else 'Short'
    w='Heavy' if r['Weight']>=80 else 'Light'
    return t+' and '+w
df['Build']=df.apply(cat, axis=1)
```

C7 **7. Cities hosting Winter Olympics football between 1930 and 2010.**

Filter season/sport/year window, take unique City.

```
m=df[(df['Season']=='Winter') & (df['Sport']=='Football') &
     (df['Year'].between(1930,2010))]
print(m['City'].unique())
```

C8 **8. Function: given a player name, return unique countries, sports, Olympics attended.**

Filter to the player, then .unique() on each column.

```
def player_info(name):
    p=df[df['Name']==name]
    return p['region'].unique(), p['Sport'].unique(), p['Games'].unique()
```

C9 **9. Sport with most medals in Summer and Winter separately.**

Drop NaN medals, groupby Season+Sport, count, idxmax per season.

```
won=df.dropna(subset=['Medal'])
for s in ['Summer','Winter']:
    c=won[won['Season']==s]['Sport'].value_counts()
    print(s, '->', c.idxmax())
```

March 2024 (scanned set)

100 marks · dataset: countries.csv, adult.csv

↑ Top

SECTION A — THEORY

A1 Count occurrences of a particular element in a list. a_list=['a',...]

list.count(value) returns how many times value appears.

```
a_list=['a','b','a','c']; print(a_list.count('a'))  # 2
```

A2 What is the global keyword in Python?

global lets a function modify a variable defined at module (global) scope instead of creating a local one.

```
x=0
def inc():
    global x; x+=1
```

A3 How can you get a random number in Python?

random.random()/randint from the random module.

```
import random; random.randint(1,10)
```

A4 List and dictionary comprehensions with an example.

One-line build. List: [f(x) for x in seq]; dict: {k:v for ...}.

```
[i*i for i in range(5)]
{i:i*i for i in range(5)}
```

A5 Explain negative indexing with example.

Negative indexes count from the end: -1 is last, -2 second last.

```
'abc'[-1]   # 'c'
```

A6 Which function removes duplicates in DataFrames?

drop_duplicates() removes duplicate rows; subset= limits which columns define a duplicate.

```
df.drop_duplicates()
```

A7 Identify and deal with missing values in a DataFrame.

isnull().sum() to find; dropna()/fillna() to handle.

↑ Top

```
df.isnull().sum(); df.fillna(0)
```

A8 Explain different ranking methods for Series.

rank(method=...): average, min, max, first, dense — differ in how ties are scored.

```
s.rank(method='min')
```

A9 What is GroupBy in pandas?

Split-apply-combine: split rows by a key, apply an aggregate, combine into a result.

```
df.groupby('k')['v'].sum()
```

A10 Which plot visualizes distribution of data? Which methods?

Histogram, KDE, boxplot, distplot/histplot. seaborn: histplot, kdeplot, boxplot.

```
sns.histplot(df['x']); sns.boxplot(x=df['x'])
```

SECTION B — PROGRAMMING

B1 Happy number check.

Repeatedly sum squares of digits; track seen numbers; happy if it reaches 1.

```
def is_happy(n):
    seen=set()
    while n!=1 and n not in seen:
        seen.add(n)
        n=sum(int(d)**2 for d in str(n))
    return n==1
print(is_happy(19))  # True
```

B2 Grade test scores: A>=90, B 80-89, C 70-79, D 60-69, F<60; print grade list.

Map each score to a letter via a banding function.

```
def grade(s):
    return 'A' if s>=90 else 'B' if s>=80 else 'C' if s>=70 else 'D' if s>=60 else
scores=[95,82,77,61,40]
print([grade(s) for s in scores])
```

↑ Top

B3 Validate a date: full month name + day number; check the day is valid for that month.

Dict month->max days; compare the entered day against it.

```
days={'January':31,'February':28,'March':31,'April':30,'May':31,'June':30,
      'July':31,'August':31,'September':30,'October':31,'November':30,'December':31
def valid(month, day): return month in days and 1<=day<=days[month]
print(valid('February',30))  # False
```

B4 Superstore tiered pricing (Category A/B by quantity bands) + bill discounts; loop until STOP.

Lookup unit price by category and quantity band, accumulate, then apply 5%/10% discount thresholds.

```
def unit_price(cat, q):
    if cat=='A': return 15 if q<30 else 12 if q<100 else 10
    else:       return 30 if q<50 else 25 if q<100 else 20
def bill(items):           # items: list of (cat, qty)
    total=sum(unit_price(c,q)*q for c,q in items)
    if total>1500: total*=0.90
    elif total>1000: total*=0.95
    return total
```

B5 UsernameValidation(str): 4-25 chars, starts with letter, only letters/digits/_, no trailing _.

Same validation pattern as other papers; return 'true'/'false'.

```
def UsernameValidation(s):
    ok=(4<=len(s)<=25 and s[0].isalpha()
        and s.replace('_','').isalnum() and not s.endswith('_'))
    return 'true' if ok else 'false'
```

SECTION C — DATA ANALYSIS (EDA)

C1 (countries 1) Overall life expectancy across the world.

Mean of the life-expectancy column.

```
c=pd.read_csv('countries.csv')
print(c['life_expectancy'].mean())
```

C2 (countries 2) DataFrame of 10 countries with highest populations.

Sort descending by population, head(10).

```
print(c.sort_values('population', ascending=False).head(10))
```

↑ Top

C3 (countries 3) Add overall GDP column = population * per-capita GDP.

Vectorised column multiply.

```
c['gdp']=c['population']*c['gdp_per_capita']
```

C4 (countries 4) 10 countries with lowest GDP-per-capita among population>160 million.

Filter population, sort ascending by per-capita, head(10).

```
big=c[c['population']>160_000_000]
print(big.sort_values('gdp_per_capita').head(10))
```

C5 (countries 5) Count countries per continent.

value_counts or groupby size.

```
print(c['continent'].value_counts())
```

C6 (countries 6) How does population distribution vary across continents?

Boxplot of population by continent.

```
sns.boxplot(data=c, x='continent', y='population'); plt.xticks(rotation=45); plt.sh
```

C7 (countries 7) Categorize by life expectancy using apply -> new column.

apply a banding function.

```
def le(v): return 'Low' if v<60 else 'Medium' if v<75 else 'High'
c['LE_Cat']=c['life_expectancy'].apply(le)
```

C8 (countries 8) Average population per continent where avg life expectancy > 75.

groupby continent; filter groups by mean LE; take mean population.

```
g=c.groupby('continent')
mask=g['life_expectancy'].mean()>75
print(g['population'].mean()[mask])
```

C9 (adult 1) Proportion of German citizens (native-country).

Value count of Germany / total.

↑ Top

```
a=pd.read_csv('adult.csv')
print((a['native-country']=='Germany').mean())
```

C10 (adult 2) Most common occupation.

mode / value_counts top.

```
print(a['occupation'].mode()[0])
```

C11 (adult 3) Mean & std of age for >50K vs <=50K earners.

groupby salary, agg mean+std of age.

```
print(a.groupby('salary')['age'].agg(['mean','std']))
```

C12 (adult 4) Do >50K earners always have at least high-school education?

Check which education levels appear among >50K earners.

```
hi=a[a['salary']=='>50K']
print(hi['education'].unique())
```

C13 (adult 5) Among married vs single men, who has higher proportion of >50K?

Filter Male; group by marital status family; compare >50K share.

```
men=a[a['sex']=='Male']
men=men.assign(married=men['marital-status'].str.startswith('Married'))
print(men.groupby('married').apply(lambda g:(g['salary']=='>50K').mean()))
```

C14 (adult 6) Max hours-per-week; how many work that; % earning >50K among them.

max, count of max, share of >50K within them.

```
mx=a['hours-per-week'].max()
sub=a[a['hours-per-week']==mx]
print(mx, len(sub), (sub['salary']=='>50K').mean())
```

C15 (adult 7) Average work hours for low vs high earners per country.

groupby country+salary, mean of hours.

```
print(a.groupby(['native-country','salary'])['hours-per-week'].mean())
```

↑ Top

August 2021 80 marks · dataset: covid.csv

SECTION A — THEORY

A1 Immutable built-in datatypes; demonstrate immutability with code.

Immutable: int, float, bool, str, tuple, frozenset, bytes. Reassigning makes a new object.

```
t=(1,2)
# t[0]=9  -> TypeError
print(id(t)); t=t+(3,); print(id(t))  # new id
```

A2 Explain negative indexing.

Index from the end: -1 last element, -2 second last.

```
[10,20,30][-1]   # 30
```

A3 What is a string; explain slicing.

Immutable character sequence. Slice s[start:stop:step].

```
'hello'[1:4]   # 'ell'
```

A4 What are map and reduce functions?

map transforms each element; reduce folds to one value.

```
from functools import reduce; reduce(lambda a,b:a*b,[1,2,3,4])  # 24
```

A5 Is Python statically or dynamically typed? Explain.

Dynamically typed: types are bound at runtime, no declarations; a name can be rebound to another type.

```
x=5; x='now a string'   # legal
```

A6 Significant features of pandas.

DataFrame/Series structures, label-based indexing, easy I/O (CSV/Excel/SQL), groupby, missing-data handling, time series, merging/joining.

↑ Top

A7 Explain categorical data in pandas.

The category dtype stores repeated text labels efficiently and supports ordering; saves memory and speeds groupby.

```
df['c']=df['c'].astype('category')
```

A8 Difference between matrices and arrays.

np.array is N-dimensional and the general workhorse; np.matrix is strictly 2-D with * meaning matrix multiply. Arrays are preferred.

```
np.array([[1,2],[3,4]])
```

A9 Create a multi-dimensional array from a 1-D array.

Use reshape to change the shape without changing data.

```
np.arange(6).reshape(2,3)
```

A10 Difference between sort_values() and sort_index() for a Series.

sort_values orders by the values; sort_index orders by the index labels.

```
s.sort_values(); s.sort_index()
```

SECTION B — PROGRAMMING

B1 Nested list: find minimum element and average of all elements. list1=[[20,25,30],24,...]

Flatten (handle ints and sublists), then min() and mean.

```
list1=[[20,25,30],24,56,[10,15,18],[12,45,20],35,20,23,28]
flat=[]
for x in list1:
    flat += x if isinstance(x,list) else [x]
print('min:', min(flat))
print('avg:', sum(flat)/len(flat))
```

B2 Job-fair scheduling: pick max non-overlapping company slots (greedy by end time).

Classic activity-selection: sort by end time, greedily pick each slot that starts after the last chosen end.

```
companies=[('DataVision',8.0,9.0),('InfoWorld',8.0,9.5),('SkyView',11.0,11.5)]
companies.sort(key=lambda c:c[2])     # sort by end time
chosen=[]; last_end=0
for name,s,e in companies:
    if s>=last_end:
```

↑ Top

```
        chosen.append(name); last_end=e
print(chosen)
```

B3 **Verify every number in a list is even, using user-defined functions and map-reduce.**

map each to a boolean, reduce with AND.

```
from functools import reduce
def is_even(x): return x%2==0
def all_even(nums): return reduce(lambda a,b:a and b, map(is_even,nums), True)
print(all_even([2,4,6]))   # True
```

SECTION C — DATA ANALYSIS (EDA)

C1 **(a1) Convert columns ['cp','fbs','restecg','keratin_type','Immunity'] to categorical.**

astype('category') on the listed columns.

```
df=pd.read_csv('covid.csv')
for col in ['cp','fbs','restecg','keratin_type','Immunity']:
    df[col]=df[col].astype('category')
```

C2 **(a2) Drop the patient id column.**

drop with axis=1.

```
df=df.drop(columns=['pid'])
```

C3 **(a3) Visualize Age distribution over blood_group; comment.**

Boxplot Age by blood group.

```
sns.boxplot(data=df,x='blood_grp',y='Age'); plt.show()
```

C4 **(a4) Visualize relationship between Age and thalachh; comment.**

Scatter plot.

```
sns.scatterplot(data=df,x='Age',y='thalachh'); plt.show()
# Comment: max heart rate tends to fall with age.
```

C5 **(a5) Unique values in Addiction; how many different types.**

unique() and nunique().

↑ Top

```
print(df['Addiction'].unique(), df['Addiction'].nunique())
```

C6 **(b1) Cross table Diabetes type vs survived; survival % per diabetes type; inference.**

> crosstab then row-normalize to percentages.

```
ct=pd.crosstab(df['Diabetestype'], df['Survive_status'])
pct=ct.div(ct.sum(axis=1), axis=0)*100
print(pct)
```

C7 **(b2) Average hemoglobin for male/female; count of each sex with hemoglobin<10.**

> groupby Sex mean; filter then count by sex.

```
print(df.groupby('Sex')['Hemoglobin'].mean())
print(df[df['Hemoglobin']<10].groupby('Sex').size())
```

C8 **(b3) Distribution of trtbps, 20 bins.**

> histplot with bins=20.

```
sns.histplot(df['trtbps'], bins=20); plt.show()
```

C9 **(b4) Distribution of chol, 20 bins; comment.**

> histplot bins=20; comment on skew.

```
sns.histplot(df['chol'], bins=20); plt.show()
```

July 2021

80 marks · dataset: website ratings dataset

SECTION A — THEORY

A1 **Difference between List and Tuple.**

> List mutable [], tuple immutable (); tuple hashable and slightly faster.

```
[1,2].append(3); # (1,2) cannot change
```

↑ Top

A2 Difference between index() and find() for strings.

Both locate a substring; find() returns -1 if absent, index() raises ValueError.

```
'abc'.find('z')   # -1
# 'abc'.index('z') # error
```

A3 What is type casting in Python?

Explicit conversion of type with int(), float(), str(), etc.

```
int('5')+2   # 7
```

A4 How to change the value of a key in a dictionary?

Assign to the key directly.

```
d={'a':1}; d['a']=99
```

A5 What are anonymous functions?

Lambdas: nameless one-expression functions.

```
(lambda x:x+1)(4)   # 5
```

A6 What is an identity matrix; create it with numpy.

Square matrix with 1s on the diagonal, 0 elsewhere.

```
np.eye(3)
```

A7 Explain split() for arrays.

np.split divides an array into multiple sub-arrays along an axis.

```
np.split(np.arange(6), 3)   # 3 arrays
```

A8 Explain reshape().

Returns a new view with a different shape; total elements must match.

```
np.arange(6).reshape(2,3)
```

↑ Top

A9 Difference between sort_values() and sort_index().

By data vs by index labels.

```
s.sort_values(); s.sort_index()
```

A10 Which plot visualizes distribution? Which methods?

Histogram / KDE / boxplot; seaborn histplot, kdeplot, boxplot.

```
sns.histplot(df['x'])
```

SECTION B — PROGRAMMING

B1 Computer assembly bill: price list of parts, take user choices, add 12% GST.

Nested dict of part->type->price; look up choices; total then GST = total*0.12.

```
prices={'HDD':{'1TB':5000,'2TB':7500,'4TB':10000},
        'RAM':{'8GB':4000,'16GB':6000},
        'Processor':{'I5':15000,'I7':18000}}
chosen={'HDD':'1TB','RAM':'16GB','Processor':'I5'}
total=sum(prices[item][typ] for item,typ in chosen.items()) + 4000
gst=total*0.12
print('Total with GST:', total+gst)
```

B2 Fractional knapsack: maximize profit within weight limit.

Greedy by profit/weight ratio; take whole items then a fraction of the next.

```
def knapsack(profit, weight, cap):
    items=sorted(zip(profit,weight), key=lambda x:x[0]/x[1], reverse=True)
    taken=[0]*len(items); earned=0
    for i,(p,w) in enumerate(items):
        if cap>=w: taken[i]=1; cap-=w; earned+=p
        elif cap>0: taken[i]=cap/w; earned+=p*(cap/w); cap=0
    return taken, earned
print(knapsack([1,2,3,4],[6,3,8,10],10))
```

B3 Series 0,1,3,6,10,15,...; sum of terms from k to l; accept n,k,l.

Triangular numbers: term i = i*(i+1)//2. Build, then slice-sum.

```
def series(n): return [i*(i+1)//2 for i in range(n)]
def sum_range(n,k,l):
```

↑ Top

```
    s=series(n); return sum(s[k:l+1])
print(series(11)); print(sum_range(11,2,5))
```

SECTION C — DATA ANALYSIS (EDA)

C1 (a1) Read meta-data: head(10), dtypes, count numeric vs non-numeric, statistical summary.

The standard inspection quartet.

```
df=pd.read_csv('ratings.csv')
print(df.head(10)); print(df.dtypes)
num=df.select_dtypes('number').shape[1]
print('numeric:', num, 'non-numeric:', df.shape[1]-num)
print(df.describe())
```

C2 (a2) avg_rating column = mean of Ucredit,Ureview,Web_review, rounded to 1 dp.

Row mean of three columns, round, assign.

```
df['avg_rating']=df[['Ucredit','Ureview','Web_review']].mean(axis=1).round(1)
```

C3 (a3) Count distinct products purchased/rated across product_1/2/3.

Stack the three product columns, take nunique.

```
prods=pd.concat([df['Product_1'],df['product_2'],df['product_3']])
print(prods.nunique())
```

C4 (a4) Boxplot for Web_review and Exp_review.

Boxplot of the two columns.

```
sns.boxplot(data=df[['Web_review','Exp_review']]); plt.show()
```

C5 (b1) Which metric is used for most ratings?

mode / value_counts top of assigned_metric.

```
print(df['assigned_metric'].value_counts().idxmax())
```

C6 (b2) Users with consecutive_usage > 4; average user review of that group.

Filter then mean.

↑ Top

```
grp=df[df['consecutive_usage']>4]
print(grp['Ureview'].mean())
```

C7 **(b3) Heatmap of correlation among 5 numeric columns.**

corr() on the subset, heatmap with annot.

```
cols=['Ucredit','Ureview','Web_review','consecutive_usage','Exp_review']
sns.heatmap(df[cols].corr(), annot=True, cmap='coolwarm'); plt.show()
```

C8 **(b4) Count-plot for assigned ratings.**

countplot of assigned_rating.

```
sns.countplot(data=df, x='assigned_rating'); plt.show()
```

C9 **(b5) Distribution of avg_rating, 30 bins.**

histplot bins=30.

```
sns.histplot(df['avg_rating'], bins=30); plt.show()
```

Model Set

100 marks · dataset: student stress dataset

SECTION A — THEORY

A1 **Demonstrate 'Python is case sensitive'.**

Identifiers differing only by case are different names.

```
Var=1; var=2; print(Var, var)   # 1 2
```

A2 **Two examples of implicit type casting.**

Python auto-widens types in mixed operations.

```
print(2 + 3.0)     # int->float = 5.0
print(True + 1)    # bool->int = 2
```

↑ Top

A3 What is a complex data type in Python?

complex holds real+imaginary parts, written a+bj.

```
z=2+3j; print(z.real, z.imag)
```

A4 What is a default parameter in a function?

A parameter with a fallback value used when no argument is passed.

```
def greet(name='Guest'): print('Hi', name)
greet()
```

A5 Arithmetic operators usable with strings.

+ concatenates, * repeats.

```
'ab'+'cd'   # 'abcd'
'ab'*3       # 'ababab'
```

A6 Significant features of NumPy.

ndarray, vectorized math, broadcasting, linear algebra, random module, fast C-backed operations.

A7 Difference between pivot table and cross table.

pivot_table aggregates a value with any function; crosstab counts frequencies.

```
pd.crosstab(df.a, df.b)
```

A8 Explain the random package of NumPy.

np.random provides rand, randint, randn, choice, shuffle, seed for reproducibility.

```
np.random.seed(0); np.random.rand(3)
```

A9 How to delete rows and columns from a DataFrame?

drop with axis=0 for rows (by index) and axis=1 for columns (by name).

```
df.drop(index=0); df.drop(columns=['c'])
```

↑ Top

A10 How to rank rows of a DataFrame?

rank() on a column, choosing a tie method.

```
df['r']=df['score'].rank(ascending=False)
```

SECTION B — PROGRAMMING

B1 **List ops on ['Ann','Pat','David','Tisha','Sumantha']: lengths; names<5; dict name->len; sort by length desc; most frequent char.**

Comprehensions for the first three; sorted(key=len, reverse) for fourth; Counter for the fifth.

```
names=['Ann','Pat','David','Tisha','Sumantha']
lengths=[len(n) for n in names]
short=[n for n in names if len(n)<5]
d={n:len(n) for n in names}
sorted_desc=sorted(names, key=len, reverse=True)
from collections import Counter
freq=Counter(''.join(names)).most_common(1)[0][0]
print(lengths, short, d, sorted_desc, freq)
```

B2 **Farmer truck (knapsack-style): max revenue within 500kg, honoring must-sell minimums.**

Reserve must-sell quantities first, then fill remaining capacity greedily by price-per-kg.

```
veg={'Tomato':(150,500,50),'Potato':(200,420,70),'Garlic':(250,700,70),'Brinjal':(1
# (yield_kg, price_per_10kg, must_sell_kg)
cap=500; load={}
for v,(_,_,must) in veg.items(): load[v]=must; cap-=must
order=sorted(veg, key=lambda v: veg[v][1]/10, reverse=True)  # price per kg
for v in order:
    avail=veg[v][0]-load[v]
    take=min(avail, cap); load[v]+=take; cap-=take
print(load)
```

B3 **From vehicle plates, output dict number->even/odd.**

Extract the trailing digits, test parity.

```
import re
plates=['MH 12 XJ-2234','UP04LG -2455','GJ34RV- 2442','KL 07AP-2433']
out={}
for p in plates:
    num=int(re.findall(r'(\d+)$', p.replace(' ',''))[0])
    out[num]='even' if num%2==0 else 'odd'
print(out)
```

↑ Top

SECTION C — DATA ANALYSIS (EDA)

C1 (a1) % of stressed students.

Mean of the binary Stress column * 100.

```
df=pd.read_csv('stress.csv')
print(df['Stress'].mean()*100)
```

C2 (a2) Average age of female students.

Filter sex=='F', mean of age.

```
print(df[df['sex']=='F']['age'].mean())
```

C3 (a3) avg_grade column = mean of G1,G2,G3.

Row mean of the three grades.

```
df['avg_grade']=df[['G1','G2','G3']].mean(axis=1)
```

C4 (a4) Correlation between studytime and avg_grade.

corr between the two columns.

```
print(df['studytime'].corr(df['avg_grade']))
```

C5 (a5) Distribution of avg_grade.

histplot.

```
sns.histplot(df['avg_grade'], bins=20); plt.show()
```

C6 (b1) Crosstab famrel vs Stress.

crosstab of the two.

```
print(pd.crosstab(df['famrel'], df['Stress']))
```

C7 (b2) Pivot table: stressed-student count by famsize and famsup.

pivot_table with sum of Stress.

↑ Top

```
print(pd.pivot_table(df,index='famsize',columns='famsup',values='Stress',aggfunc='s
```

C8 **(b3) Min and max avg_grade for students whose parents have higher education.**

> Filter Medu==4 or Fedu==4, agg min/max.

```
he=df[(df['Medu']==4) | (df['Fedu']==4)]
print(he['avg_grade'].min(), he['avg_grade'].max())
```

C9 **(b4) Avg free time for famsize='GT3'; among those, count stressed students with freetime < that average.**

> Filter family size, compute mean freetime, count stressed below it.

```
big=df[df['famsize']=='GT3']
avg_ft=big['freetime'].mean()
print(avg_ft, len(big[(big['freetime']<avg_ft) & (big['Stress']==1)]))
```

C10 **(b5) working_mother column (yes/no); plot its impact on stress count.**

> Map Mjob to working/not, then countplot with hue=Stress.

```
df['working_mother']=df['Mjob'].apply(lambda j:'no' if j=='at_home' else 'yes')
sns.countplot(data=df, x='working_mother', hue='Stress'); plt.show()
```

C11 **(c1) Boxplot of absences by gender.**

> boxplot absences vs sex.

```
sns.boxplot(data=df, x='sex', y='absences'); plt.show()
```

C12 **(c2) Pie chart of father's occupations.**

> value_counts then pie.

```
df['Fjob'].value_counts().plot(kind='pie', autopct='%1.1f%%'); plt.show()
```

C13 **(c3) How many students improve across G1<G2<G3.**

> Boolean mask on the chain.

```
imp=df[(df['G1']<df['G2']) & (df['G2']<df['G3'])]
print(len(imp))
```

↑ Top

C14 **(c4) Students in romantic relationships who improve; gender-wise distribution.**

> Add romantic filter, group by sex.

```
r=df[(df['romantic']=='yes') & (df['G1']<df['G2']) & (df['G2']<df['G3'])]
print(r.groupby('sex').size())
```

Generated from your 9 uploaded PDS ESA papers (2021–2025). Study aid based on past-paper patterns; verify dataset column names in the actual exam.

↑ Top